

What is Method Overloading?

Method overloading allows you to use the **same name** for multiple methods, provided they have **different parameter lists**. This allows a method to perform similar tasks on different data types or quantities.

```
int add(int a, int b)
double add(double a, double b)
```

Static Implementation

Java determines which version of the method to call at **Compile-Time** based on the argument data types passed by the programmer.

```
static int plusMethod(int x, int y) {
    return x + y;
}

static double plusMethod(double x, double y) {
    return x + y;
}

// Execution:
System.out.println(plusMethod(8, 5));      // Calls int version
System.out.println(plusMethod(4.3, 6.2)); // Calls double version
```

Overloading Strategies

- **By Argument Count:** Summing two numbers vs. summing three numbers.
- **By Data Type:** Calculating area with integers vs. high-precision doubles.
- **By Parameter Order:** (String, int) vs. (int, String).

Real-Life Scenario: Information Displays

Overloading is perfect for displaying user info where part of the data might be optional.

```
static void display(String name) {  
    System.out.println("User: " + name);  
}  
  
static void display(String name, int age) {  
    System.out.println("Name: " + name + " | Age: " + age);  
}
```

⚠ CRITICAL RULE: You CANNOT overload a method by changing only the return type. The parameters MUST be different.

Fast Summary

- Improves code **Readability** and **Maintainability**.
- Reduces the need to invent complex, unique names for similar actions.
- Is a form of Static Polymorphism.